INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

DISEÑO Y DESARROLLO DE SOFTWARE

CURSO: DESARROLLO DE SOLUCIONES EN LA NUBE

PROYECTO:

LeGrandHotel – Sistema web de gestión de hospedaje

Avances desde la entrega del 25%

Recorrido por pantalla y pruebas de carga

AREQUIPA, 2026

Docente:

VÍCTOR CALVO

Presentado por:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Perú

2026

Índice

1. Objetivo del documento 3
2. Qué se agregó desde la entrega del 25% 3
3. Recorrido por pantalla 5
3.1 Panel 5
3.2 Reservas 6
3.3 Estadías 7
3.4 Habitaciones 7
3.5 Tipos de habitación 8
3.6 Clientes 9
3.7 Cochera 10
3.8 Inventario 11
3.9 Personal y usuarios 12
3.10 Reportes 13
3.11 Auditoría 14
3.12 Vista del programador 15
3.13 Configuración 16
4. Las pantallas del cliente 18
4.1 Reservar una habitación 18
4.2 Mis reservas 18
4.3 Mi perfil 19
5. Lo nuevo, en detalle 20
5.1 Tutorial de bienvenida y tutorial de cada pantalla 20
5.2 Las reglas de validación a la vista 21
5.3 Personal y Usuarios en una sola pestaña 22
5.4 Filtros del tablero de habitaciones 23
5.5 Despliegue en Render 23
6. Pruebas de carga 24
6.1 Cómo se midió y por qué cambió el método 24
6.2 Prueba de estrés: 100 clientes reservando sin pausa 25
6.3 Prueba de resistencia: 5000 usuarios entrando en bloques de 100 27
6.4 Dónde está el techo 28
7. Caídas e incidentes del periodo 30
8. Conclusiones y pendientes 31
Pendientes 31

1. Objetivo del documento

Este documento registra todo lo que se hizo en LeGrandHotel desde la entrega del 25%. Tiene tres partes: un recorrido por cada pantalla del sistema, con su captura y una tabla que dice qué hace cada botón y a qué vista lleva; el detalle de lo que se agregó o cambió en ese periodo; y las dos últimas pruebas de carga, con sus gráficos e interpretación.

La aplicación está desplegada y funcionando en Render, con la API en Docker sobre PostgreSQL y la web como sitio estático. Las pruebas de carga se hicieron en la laptop del equipo, porque el servidor en AWS todavía no está creado; por eso sus resultados muestran el límite de una sola máquina y no el del sistema desplegado.

2. Qué se agregó desde la entrega del 25%

La tabla resume el trabajo del periodo. La columna «Dónde» indica el archivo o la carpeta donde se puede verificar cada punto.

Tabla 1. Trabajo realizado desde el 27/10/2026

Qué se hizo	En qué consiste	Dónde	Tipo
Despliegue en Render	La API y la web funcionando en internet: API en Docker con PostgreSQL, web como sitio estático, migraciones al arrancar y siembra opcional de datos de demostración	render.yaml, api/Dockerfile.render, docs/despliegue-render.md	Evaluado 80%
Tutorial de bienvenida	Al primer ingreso se ofrece un recorrido que oscurece la pantalla y resalta cada sección según el rol; se omite o se repite desde Configuración	Tour.tsx, tour.ts; POST /auth/tutorial	Extra
Tutorial en cada pestaña	Un botón «?» al lado del título de cada pantalla abre un recorrido que explica, uno por uno, para qué sirve cada elemento de esa pantalla	tutoriales.ts, AyudaDePantalla	Extra
Reglas de validación a la vista	Al costado de cada casilla, en letra pequeña, dice qué acepta: obligatorio, solo letras, solo números, máximos y mínimos	ui.tsx (FieldLabel), formularios	Extra
Casillas que solo aceptan lo válido	Nombre y nacionalidad solo letras; DNI, teléfono y stock solo números; la unidad del insumo se elige de una lista y no se escribe	ui.tsx (soloLetras, soloNumeros)	Extra
Contraseña visible con mínimo	El formulario de personal muestra u oculta la contraseña y exige al menos 8 caracteres con letras y números	PasswordField	Mejora
Personal y Usuarios unidos	Una sola pestaña «Personal y usuarios»; al crear se abre el formulario de personal. Se quitaron la pestaña «Registrar personal», «Facturación» y los campos Cargo y Turno	UsersPage.tsx, App.tsx	Mejora
Filtros del tablero	El tablero de habitaciones del panel filtra por piso y por estado, y muestra las 70 habitaciones (antes 50)	RoomRack.tsx; rack.test.tsx	Extra
Vista del programador en Render	Muestra el servicio, la rama y el commit desplegados, leídos de la configuración para que funcionen con la caché de producción	config/hotel.php, DeveloperPage	Extra
Pruebas de carga dentro de Docker	Las pruebas se corren dentro de la red de Docker: antes el proxy de Docker Desktop en Windows rechazaba las conexiones y falseaba el resultado	tests/k6/dentro.sh	Evaluado 80%

Qué se hizo	En qué consiste	Dónde	Tipo
Scripts de preparación	preparar.sh deja la API lista (límites, clientes, caché) y restaurar.sh la devuelve a la normalidad	tests/k6/preparar.sh, restaurar.sh	Extra
Límite de peticiones corregido	Una variable vacía en el archivo de configuración dejaba el límite en 0 y la API respondía 429 a todo	config/hotel.php; RateLimitConfigTest	Corrección
Respaldos en GitHub	Copia de seguridad cifrada de la base en un repositorio aparte	infra/scripts/backup-mysql-to-github.sh	Evaluado 80%
Registro de caídas	Cada caída, incluso local, con su motivo y su solución (sección 7)	docs/incidentes.md	Evaluado

3. Recorrido por pantalla

Esta sección recorre el sistema pantalla por pantalla, como lo ve un administrador. De cada una se muestra la captura, para qué sirve y una tabla con cada botón: qué hace y a qué vista lleva.

Todas las pantallas comparten la misma estructura: el título con su botón principal a la derecha, las pestañas de filtro, el buscador, la tabla, los botones de cada fila y el paginado. Al lado de cada título hay un botón «?» que abre el tutorial de esa pantalla.

3.1 Panel

Para qué sirve. Es la pantalla de inicio del personal y el resumen del día: cuánto está ocupado el hotel, quién llega, quién sale y qué insumos faltan.

Dirección: `/panel`

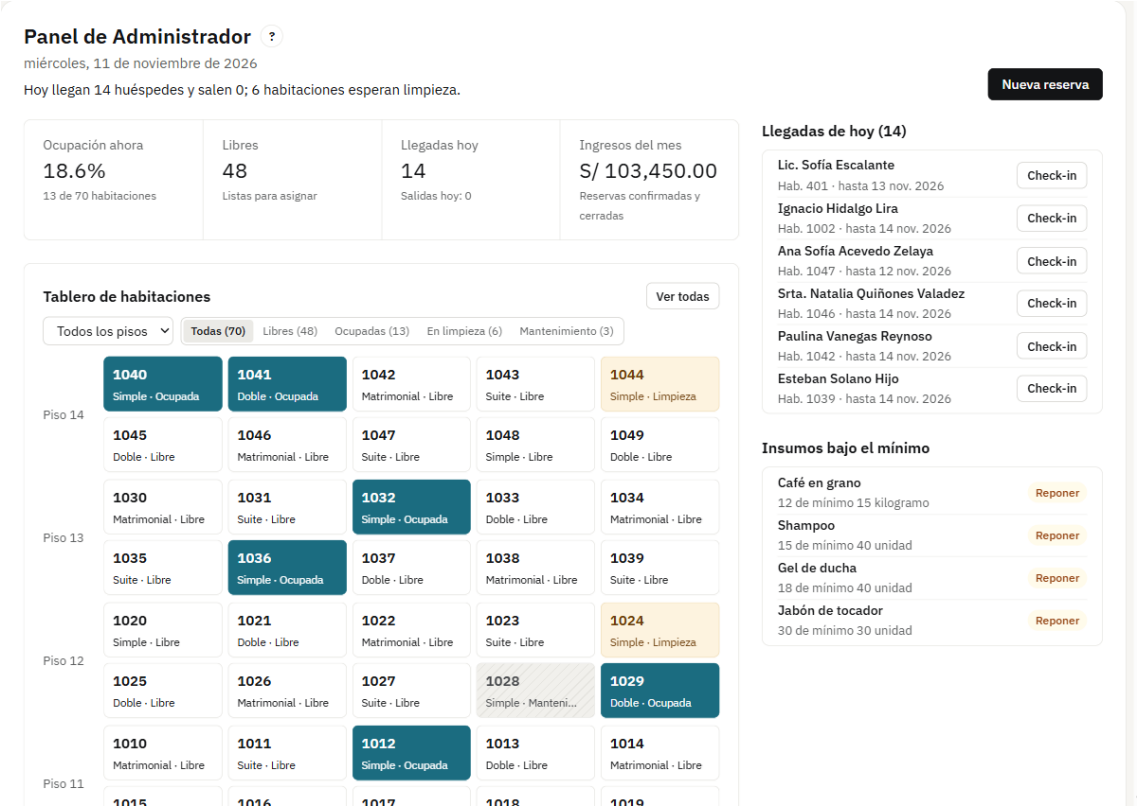


Figura 1. Panel del administrador: los cuatro indicadores, el tablero de habitaciones, las llegadas del día y los insumos bajo el mínimo

Tabla 2. Botones de Panel

Botón	Qué hace	A dónde lleva
Nueva reserva	Abre el formulario de reserva sin pasar por la pestaña Reservas	Formulario emergente sobre el Panel
Check-in (en cada llegada)	Marca que el huésped llegó y abre su estadía	La fila desaparece de «Llegadas de hoy»
Reponer (en cada insumo)	Registra una entrada del insumo que está bajo el mínimo	Formulario de movimiento
Selector de piso	Muestra solo las habitaciones de ese piso	El mismo tablero, filtrado

Botón	Qué hace	A dónde lleva
Todas / Libres / Ocupadas / En limpieza / Mantenimiento	Filtra el tablero por estado; el número entre paréntesis dice cuántas hay	El mismo tablero, filtrado
Ver todas	Abre el listado completo de habitaciones	Pestaña Habitaciones
Una habitación del tablero	Abre esa habitación para editarla	Formulario de la habitación
? (al lado del título)	Abre el tutorial de esta pantalla	Recorrido guiado sobre el Panel

3.2 Reservas

Para qué sirve. Aquí se registra quién va a venir, en qué habitación y en qué fechas. La reserva se convierte en estadía cuando el huésped llega.

Dirección: `/reservas`

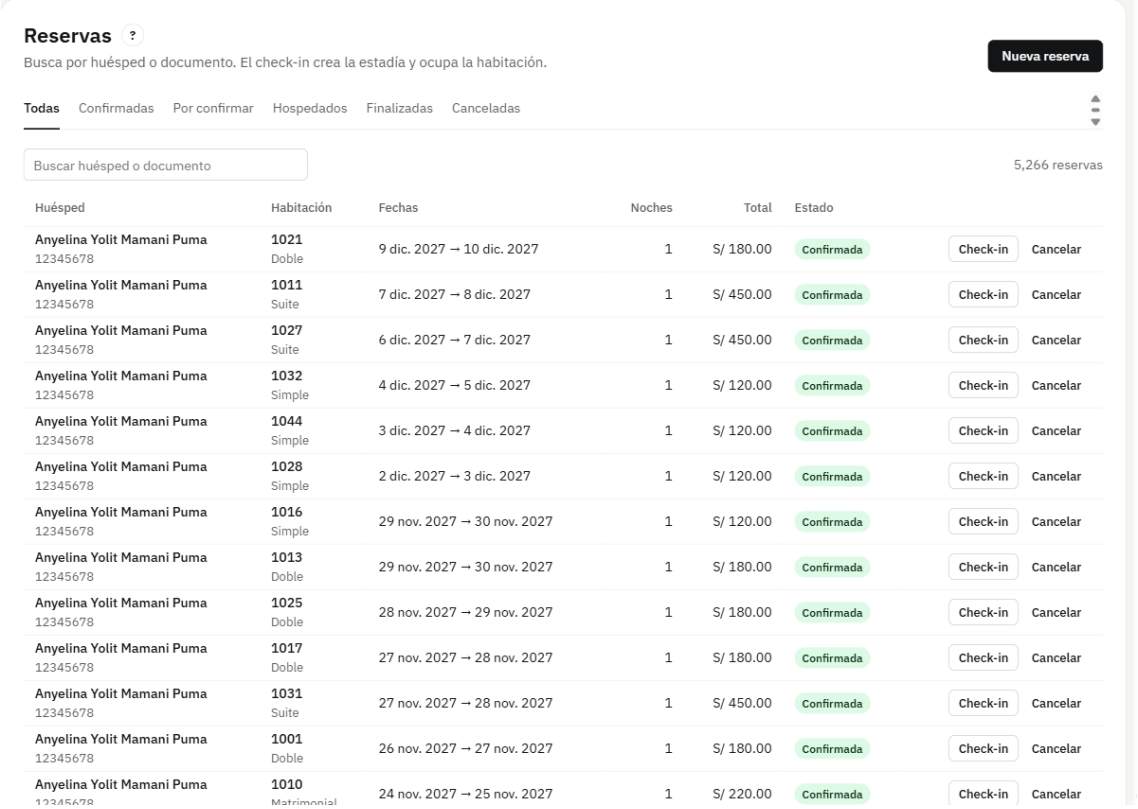


Figura 2. Reservas: listado con filtros por estado, buscador y los botones de cada fila

Tabla 3. Botones de Reservas

Botón	Qué hace	A dónde lleva
Nueva reserva	Elige huésped, fechas y una habitación libre; el total se calcula con el precio por noche	Formulario emergente
Todas / Confirmadas / Por confirmar / Hospedados / Finalizadas / Canceladas	Filtra el listado por estado	El mismo listado, filtrado
Buscar huésped o documento	Busca por nombre o número de documento	El mismo listado, filtrado

Botón	Qué hace	A dónde lleva
Check-in	Marca la llegada y crea la estadía; la habitación pasa a ocupada	Pestaña Estadías
Cancelar	Anula la reserva y libera la habitación	El mismo listado, con la reserva cancelada
Anterior / Siguiente	Avanza entre páginas de 15 reservas	El mismo listado

3.3 Estadías

Para qué sirve. Los huéspedes que ya están alojados. Una estadía nace del check-in de una reserva y termina con el check-out.

Dirección: `/estadias`

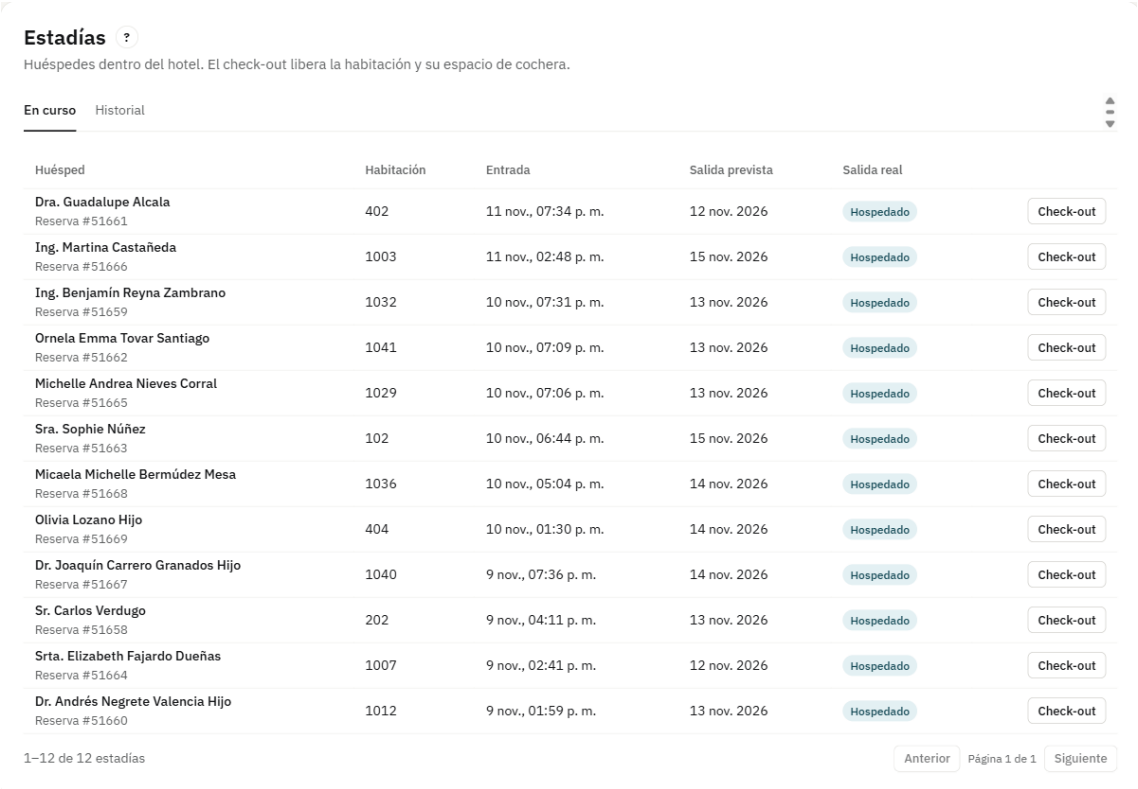


Figura 3. Estadías: los huéspedes alojados ahora y el historial

Tabla 4. Botones de Estadías

Botón	Qué hace	A dónde lleva
En curso / Historial	Cambia entre los que están ahora en el hotel y los que ya se fueron	El mismo listado, filtrado
Check-out	Cierra la estadía, libera el espacio de cochera y manda la habitación a limpieza	La estadía pasa al historial

3.4 Habitaciones

Para qué sirve. El estado de cada habitación del hotel. Al hacer check-out pasa a limpieza, y se marca libre cuando está lista.

Dirección: `/habitaciones`

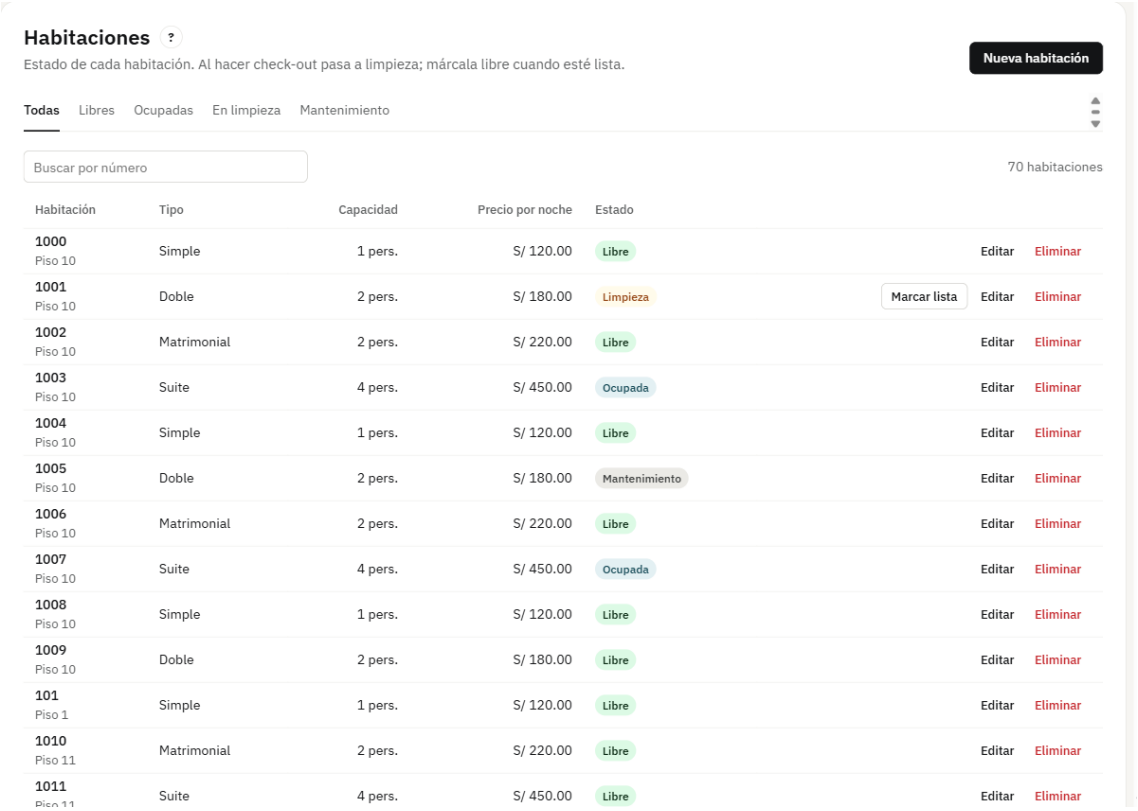


Figura 4. Habitaciones: estado de cada una, con filtros y acciones por fila

Tabla 5. Botones de Habitaciones

Botón	Qué hace	A dónde lleva
Nueva habitación	Pide número, piso, tipo y estado; el número no se puede repetir	Formulario emergente
Todas / Libres / Ocupadas / En limpieza / Mantenimiento	Filtra por estado	El mismo listado, filtrado
Marcar lista	Pasa la habitación de limpieza a libre	El mismo listado, con el estado cambiado
Editar	Cambia número, piso, tipo o estado	Formulario emergente
Eliminar	Borra la habitación, solo si no tiene reservas	El mismo listado, sin esa fila

3.5 Tipos de habitación

Para qué sirve. Las categorías del hotel (Simple, Doble, Matrimonial y Suite). De aquí salen el precio por noche, la capacidad y los servicios incluidos.

Dirección: `/tipos-habitacion`



Figura 5. Tipos de habitación: de aquí salen el precio y la capacidad de cada habitación

Tabla 6. Botones de Tipos de habitación

Botón	Qué hace	A dónde lleva
Nuevo tipo	Pide nombre, capacidad, precio por noche y servicios	Formulario emergente
Editar	Cambia el precio o los servicios; afecta solo a las reservas nuevas	Formulario emergente
Eliminar	Borra el tipo, solo si ninguna habitación lo usa	El mismo listado, sin esa fila

3.6 Clientes

Para qué sirve. Las fichas de los huéspedes. Se usan al registrar una reserva desde recepción. La columna «Cuenta web» dice si además entran por su cuenta.

Dirección: `/clientes`

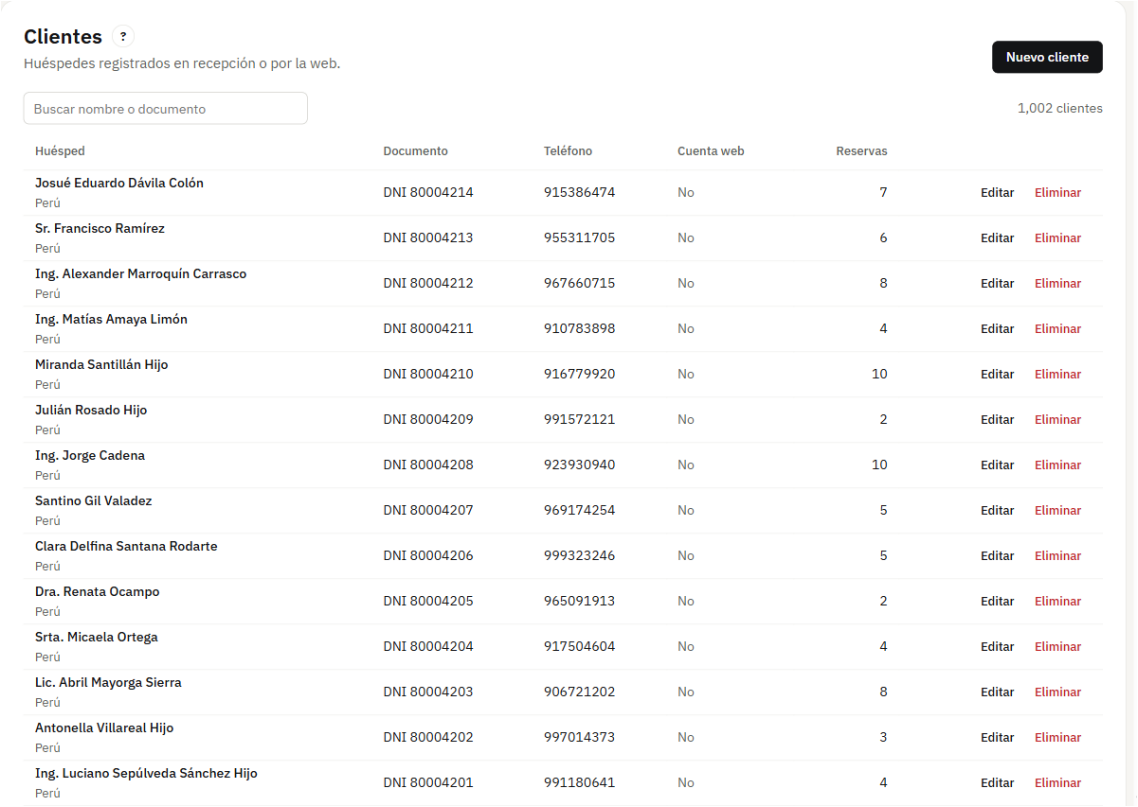


Figura 6. Clientes: las fichas de los huéspedes, con su documento, teléfono y cuántas reservas tienen

Tabla 7. Botones de Clientes

Botón	Qué hace	A dónde lleva
Nuevo cliente	Pide nombre, tipo y número de documento; el documento no se puede repetir	Formulario emergente
Buscar nombre o documento	Busca por nombre o por número de documento	El mismo listado, filtrado
Editar	Corrige los datos de la ficha	Formulario emergente
Eliminar	Borra la ficha, solo si no tiene reservas	El mismo listado, sin esa fila

3.7 Cochera

Para qué sirve. Los espacios de estacionamiento y los autos que hay dentro. El espacio se libera solo al hacer el check-out del huésped.

Dirección: `/cochera`

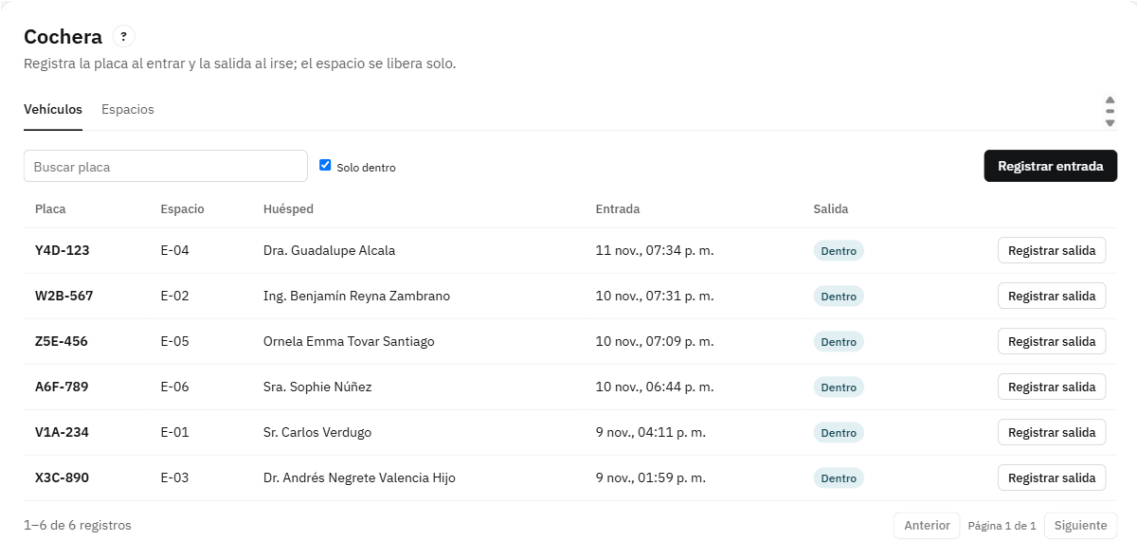


Figura 7. Cochera: los vehículos dentro y los espacios de estacionamiento

Tabla 8. Botones de Cochera

Botón	**Qué hace**	**A dónde lleva**
Vehículos / Espacios	Cambia entre los autos que están dentro y los lugares del estacionamiento	El mismo listado, en su otra vista
Registrar entrada	Anota un auto con su placa y el espacio que ocupa	Formulario emergente
Agregar espacio	Crea un lugar nuevo de estacionamiento	Formulario emergente
Registrar salida	Marca que el auto se fue y deja el espacio libre	El mismo listado, con el espacio libre

3.8 Inventario

Para qué sirve. Los insumos del hotel. El stock no se escribe a mano: cambia con los movimientos de entrada y de salida, y cada uno queda registrado con quién lo hizo.

Dirección: `/inventario`

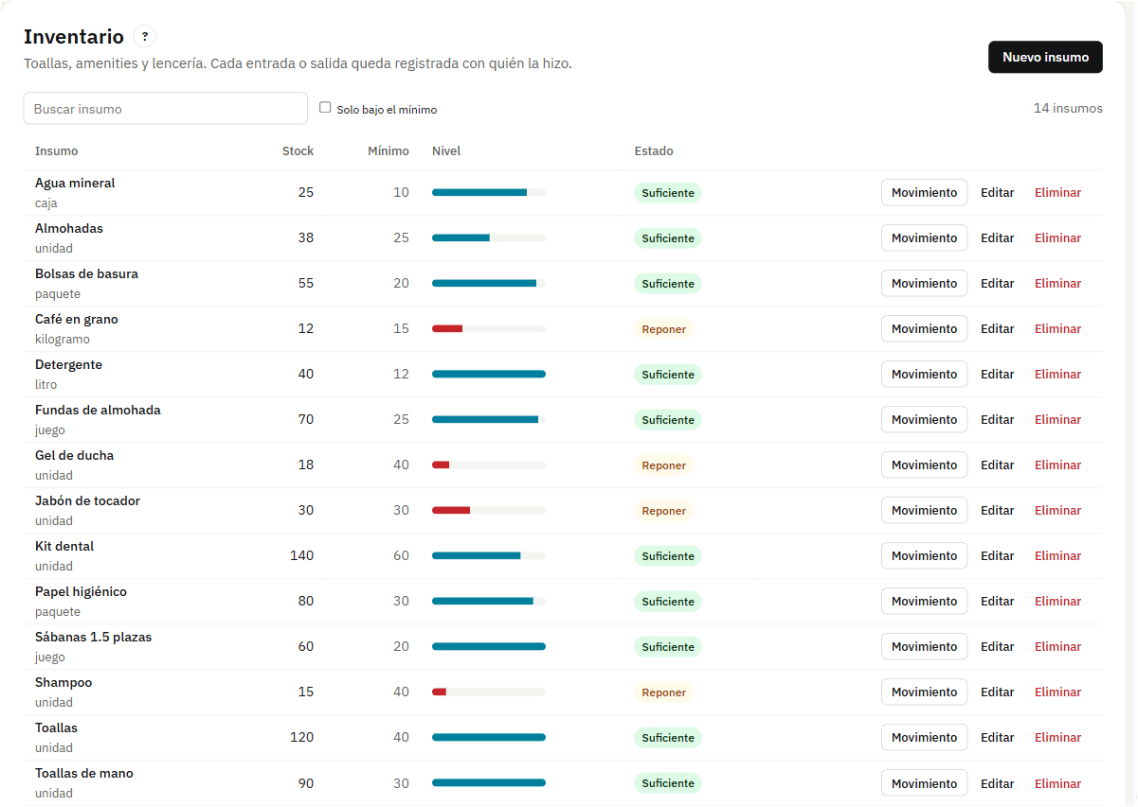


Figura 8. Inventario: los insumos del hotel y su nivel frente al mínimo

Tabla 9. Botones de Inventario

Botón	Qué hace	A dónde lleva
Nuevo insumo	Pide nombre, unidad, stock mínimo y stock inicial	Formulario emergente
Buscar insumo	Busca por nombre	El mismo listado, filtrado
Solo bajo el mínimo	Muestra únicamente los insumos que hay que reponer	El mismo listado, filtrado
Movimiento	Registra una entrada o una salida y recalcula el stock	Formulario emergente
Editar	Cambia el nombre, la unidad o el mínimo (el stock no)	Formulario emergente
Eliminar	Borra el insumo	El mismo listado, sin esa fila

3.9 Personal y usuarios

Para qué sirve. Todas las cuentas del sistema: las del equipo del hotel y las de los clientes que entran con Google. Antes eran dos pestañas separadas; ahora es una sola. Al cambiar los roles de alguien se cierran sus sesiones abiertas.

Dirección: `/usuarios`

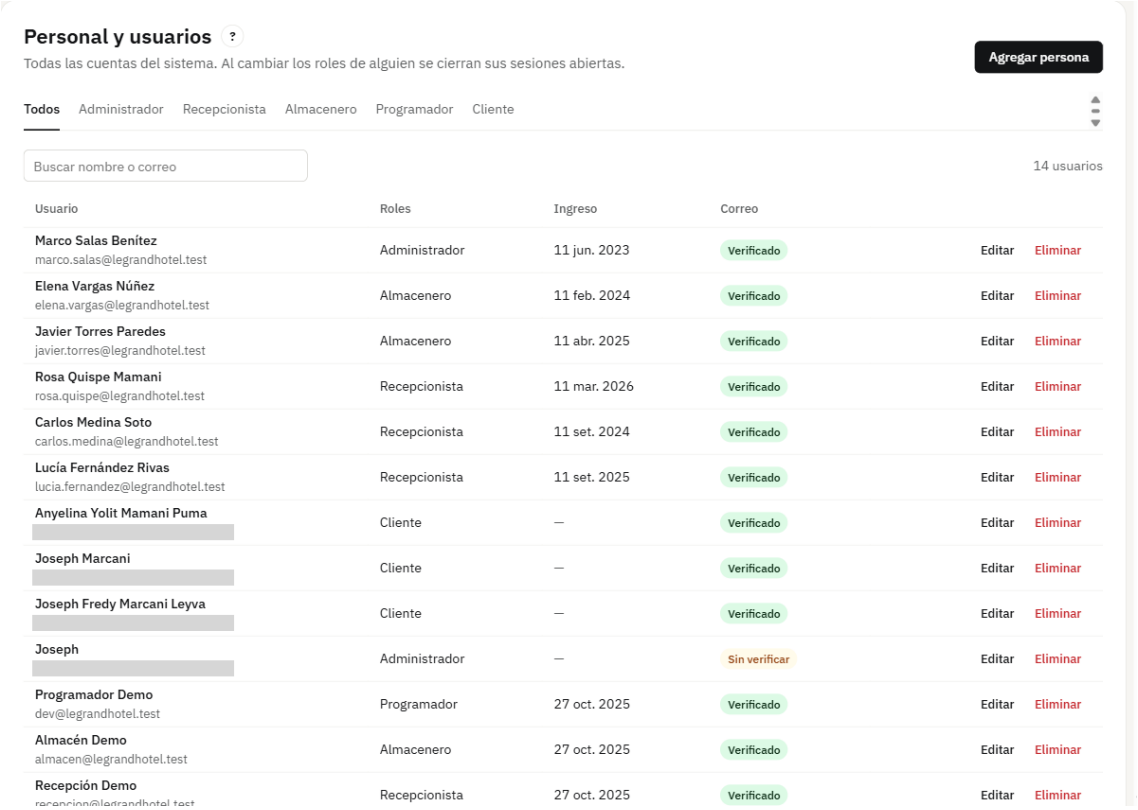


Figura 9. Personal y usuarios: una sola pestaña para el equipo del hotel y los clientes (los correos reales van tapados)

Tabla 10. Botones de Personal y usuarios

Botón	Qué hace	A dónde lleva
Agregar persona	Crea de una vez la ficha de la persona y la cuenta con la que entra; el cargo sale del rol	Formulario emergente de personal
Todos / Administrador / Recepcionista / Almacenero / Programador / Cliente	Filtra por rol	El mismo listado, filtrado
Buscar nombre o correo	Busca por nombre o por correo	El mismo listado, filtrado
Editar	Cambia nombre, correo, contraseña, roles y fecha de ingreso	Formulario emergente
Eliminar	Borra la cuenta; el Programador no se puede eliminar	El mismo listado, sin esa fila

3.10 Reportes

Para qué sirve. Dos miradas al negocio: el porcentaje de noches vendidas y los ingresos de las reservas confirmadas y cerradas, mes a mes.

Dirección: `/reportes`

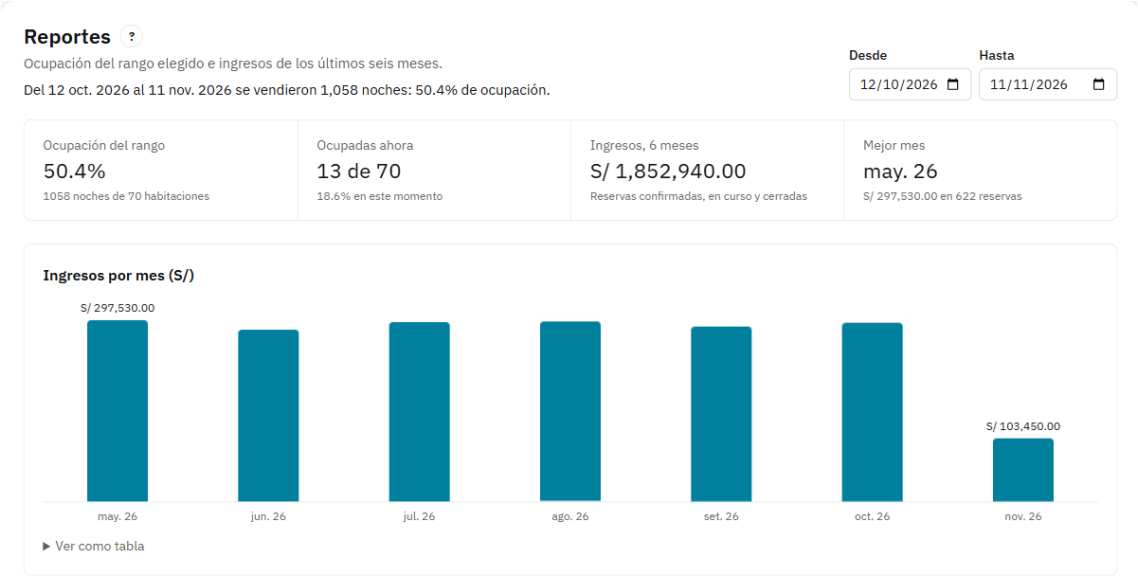


Figura 10. Reportes: ocupación e ingresos en el rango de fechas elegido

Tabla 11. Botones de Reportes

Botón	Qué hace	A dónde lleva
Desde / Hasta	Elige el rango de fechas de los dos reportes	La misma pantalla, recalculada

3.11 Auditoría

Para qué sirve. El registro de cada cambio hecho en el sistema. Sirve para rastrear un error o un cambio indebido. Es solo de lectura: no se puede borrar ni editar.

Dirección: `/auditoria`

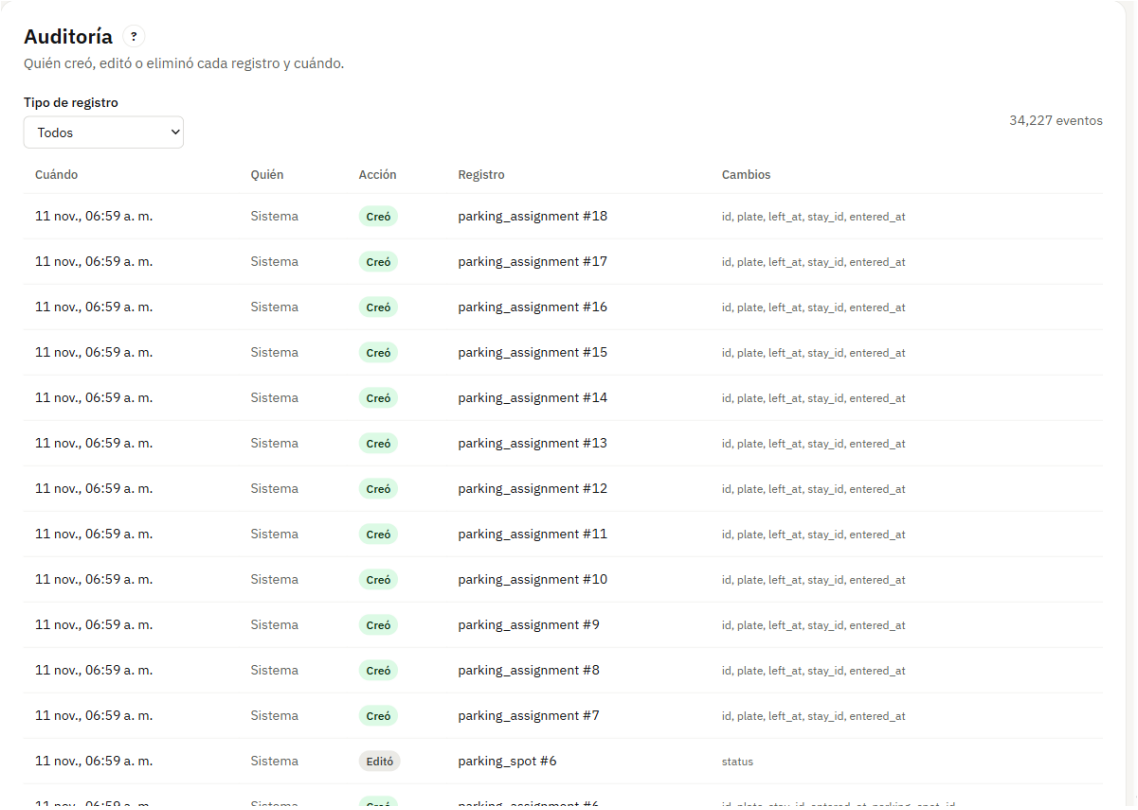


Figura 11. Auditoría: quién cambió qué y cuándo

Tabla 12. Botones de Auditoría

Botón	Qué hace	A dónde lleva
Filtros de tipo y fecha	Acota el registro por tipo de dato o por fecha	El mismo listado, filtrado
Anterior / Siguiente	Avanza entre páginas	El mismo listado

3.12 Vista del programador

Para qué sirve. El estado técnico del sistema, para revisar una falla sin entrar al servidor. Ahora muestra además el servicio, la rama y el commit desplegados en Render.

Dirección: `/programador`

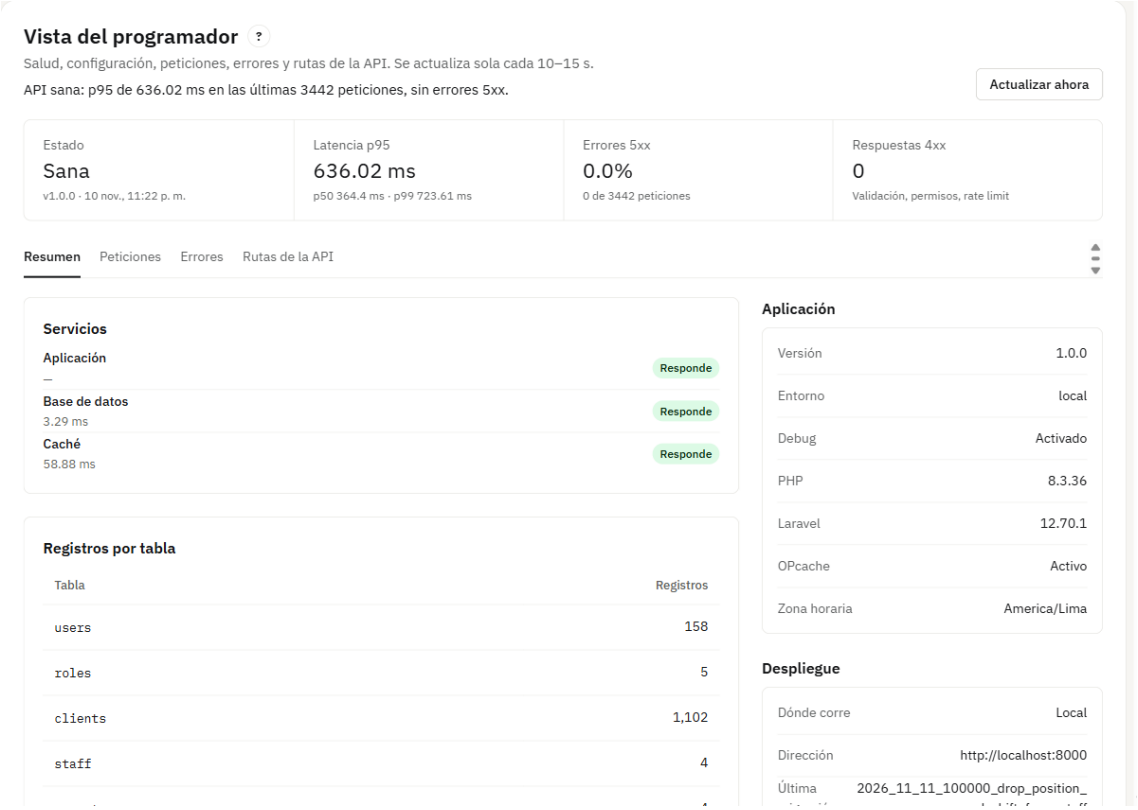


Figura 12. Vista del programador: estado técnico del sistema, ya adaptada al despliegue en Render

Tabla 13. Botones de Vista del programador

Botón	Qué hace	A dónde lleva
Resumen / Peticiones / Errores / Rutas	Cambia entre la salud del sistema, las peticiones recientes, los errores y las rutas de la API	La misma pantalla, en su otra vista

3.13 Configuración

Para qué sirve. Cómo se ve el sistema en este navegador. Los cambios se aplican al instante y solo afectan a quien los hace.

Dirección: `/configuracion`



Figura 13. Configuración: tema, idioma y el botón para repetir el tutorial

Tabla 14. Botones de Configuración

Botón	**Qué hace**	**A dónde lleva**
Claro / Oscuro / Del sistema	Cambia el tema de la interfaz	La misma pantalla, con el tema aplicado
Español / English	Cambia el idioma de la web y el de los mensajes de la API	La misma pantalla, en el otro idioma
Ver el tutorial	Vuelve a mostrar el recorrido de bienvenida	Recorrido guiado sobre la pantalla actual

4. Las pantallas del cliente

El cliente entra solo con Google y tiene su propio menú, con tres pantallas. No ve nada de la gestión del hotel. Estas tres pantallas también tienen su botón «?» con el tutorial correspondiente, agregado en este periodo.

4.1 Reservar una habitación

Para qué sirve. La pantalla con la que el cliente reserva, en tres pasos: elige sus fechas, mira lo que hay libre y reserva. Solo se muestran los tipos con sitio para el número de personas indicado.

Dirección: `/reservar`

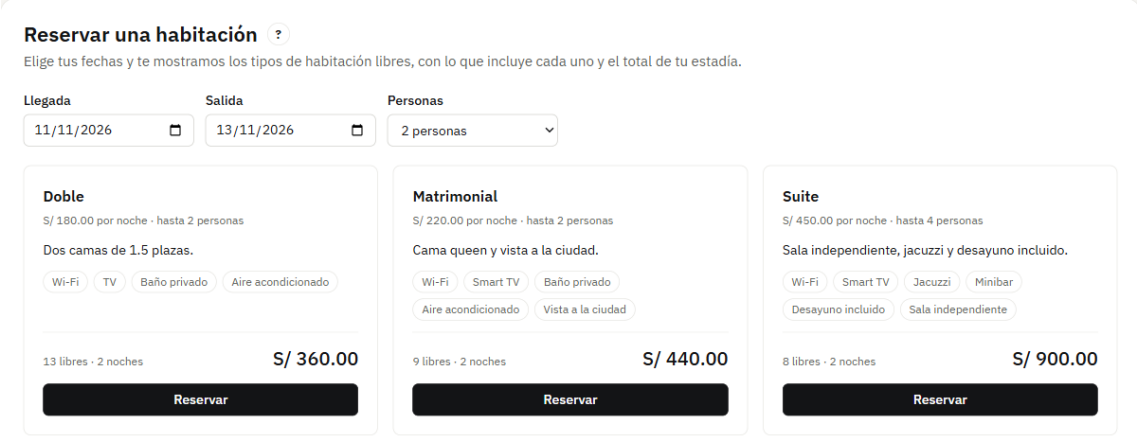


Figura 14. Reservar: el cliente elige fechas y personas, y ve los tipos libres con lo que incluye cada uno y el total de su estadía

Tabla 15. Botones de Reservar una habitación

Botón	Qué hace	A dónde lleva
Llegada / Salida / Personas	Define las fechas y cuántas personas van; la búsqueda se rehace sola	La misma pantalla, con otras ofertas
Reservar (en cada tarjeta)	Pide confirmar; si faltan el DNI o el celular los pide ahí mismo, y si ya están dice a nombre de quién queda	Confirmación y luego Mis reservas
? (al lado del título)	Abre el tutorial de cómo reservar	Recorrido guiado sobre esta pantalla

4.2 Mis reservas

Para qué sirve. Todas las reservas del cliente, con su estado y lo que va a pagar. Recepción ve exactamente lo mismo.

Dirección: `/mis-reservas`

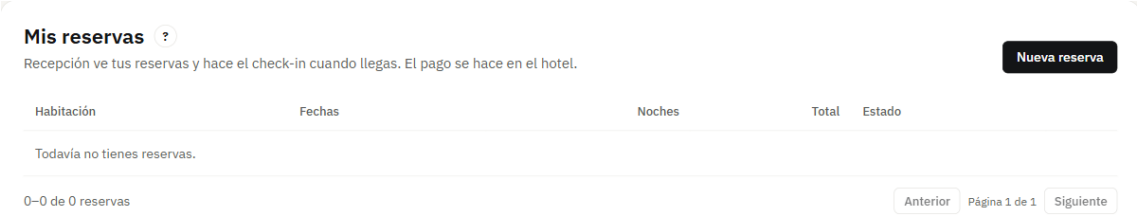


Figura 15. Mis reservas: lo que el cliente ve de cada una de sus reservas

Tabla 16. Botones de Mis reservas

Botón	Qué hace	A dónde lleva
Cancelar	Anula la reserva; pide confirmar y solo funciona antes del check-in	El mismo listado, con la reserva cancelada
Anterior / Siguiente	Avanza entre páginas	El mismo listado
? (al lado del título)	Abre el tutorial de cómo ver y cancelar reservas	Recorrido guiado sobre esta pantalla

4.3 Mi perfil

Para qué sirve. Los datos del cliente. El DNI y el celular son opcionales hasta que quiera reservar; se pueden poner aquí o en el momento de reservar. El correo viene de Google y no se puede cambiar.

Dirección: `/perfil`



Figura 16. Mi perfil: el DNI y el celular hacen falta para reservar (el correo real va tapado)

Tabla 17. Botones de Mi perfil

Botón	Qué hace	A dónde lleva
Guardar cambios	Guarda el nombre, el DNI y el celular	La misma pantalla, con el aviso de guardado
? (al lado del título)	Abre el tutorial de esta pantalla	Recorrido guiado sobre esta pantalla

5. Lo nuevo, en detalle

5.1 Tutorial de bienvenida y tutorial de cada pantalla

Son dos cosas distintas. El tutorial de bienvenida se ofrece la primera vez que alguien entra: la pantalla se oscurece y se va resaltando cada sección del menú según el rol de la persona. Se puede omitir, y se repite cuando se quiera desde Configuración. Termina señalando el botón «?», para que quede claro dónde seguir aprendiendo.

El tutorial de cada pantalla es el que abre ese botón «?», que está al lado del título en todas las vistas. Explica, uno por uno, para qué sirve cada elemento de esa pantalla concreta: el botón principal, los filtros, el buscador, la tabla, los botones de cada fila y el paginado. Si un elemento no está en esa pantalla, el recorrido lo salta.

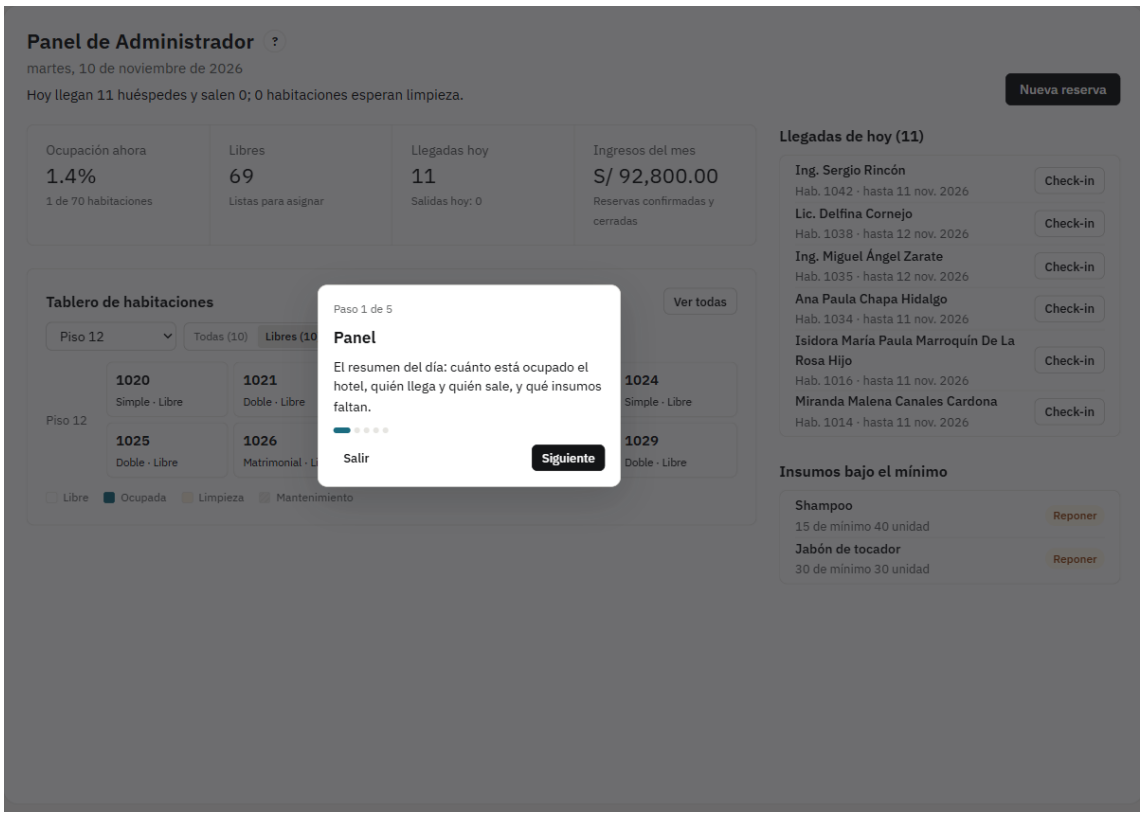


Figura 17. Primer paso del tutorial de una pantalla: la pantalla se oscurece y aparece la explicación

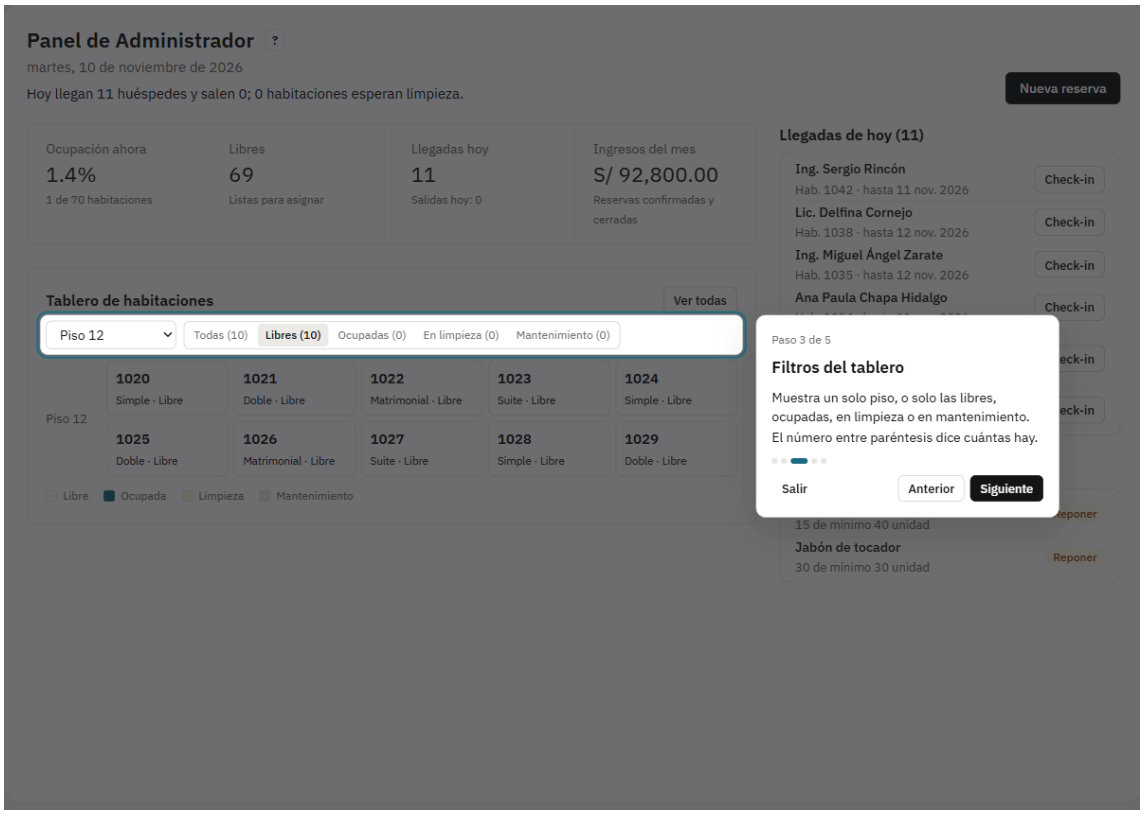


Figura 18. Tercer paso: el recorrido resalta los filtros del tablero y explica para qué sirven

5.2 Las reglas de validación a la vista

Antes, las reglas de cada casilla solo aparecían como error después de intentar guardar. Ahora están escritas en letra pequeña al costado de cada etiqueta: si el campo es obligatorio, qué acepta y cuál es el máximo o el mínimo. En el formulario de personal se ve «Obligatorio. Solo letras, hasta 120» junto a Nombre completo y «Mínimo 8 caracteres, con letras y números» junto a Contraseña.

Además, las casillas ya no dejan escribir lo que no corresponde: Nombre completo y Nacionalidad solo aceptan letras, y DNI, teléfono y los stocks solo números. La contraseña tiene un botón «Ver» para mostrarla u ocultarla, y no deja guardar si no cumple el mínimo.

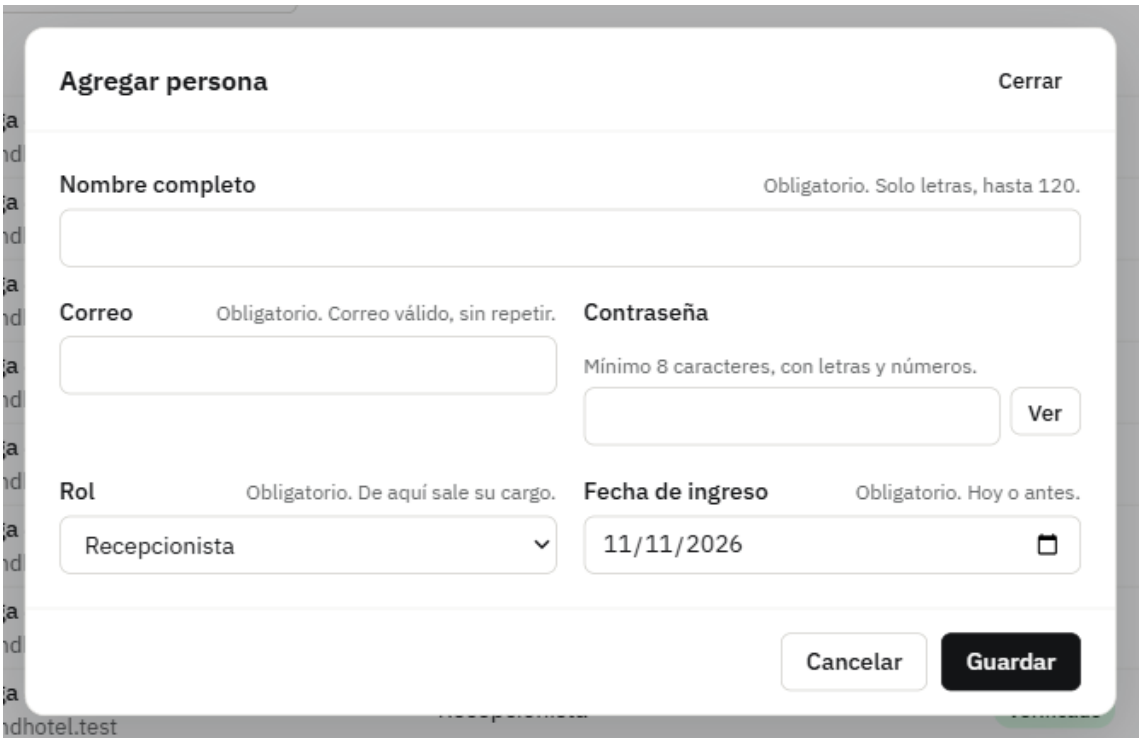


Figura 19. Agregar persona: las reglas al costado de cada casilla y el botón «Ver» de la contraseña

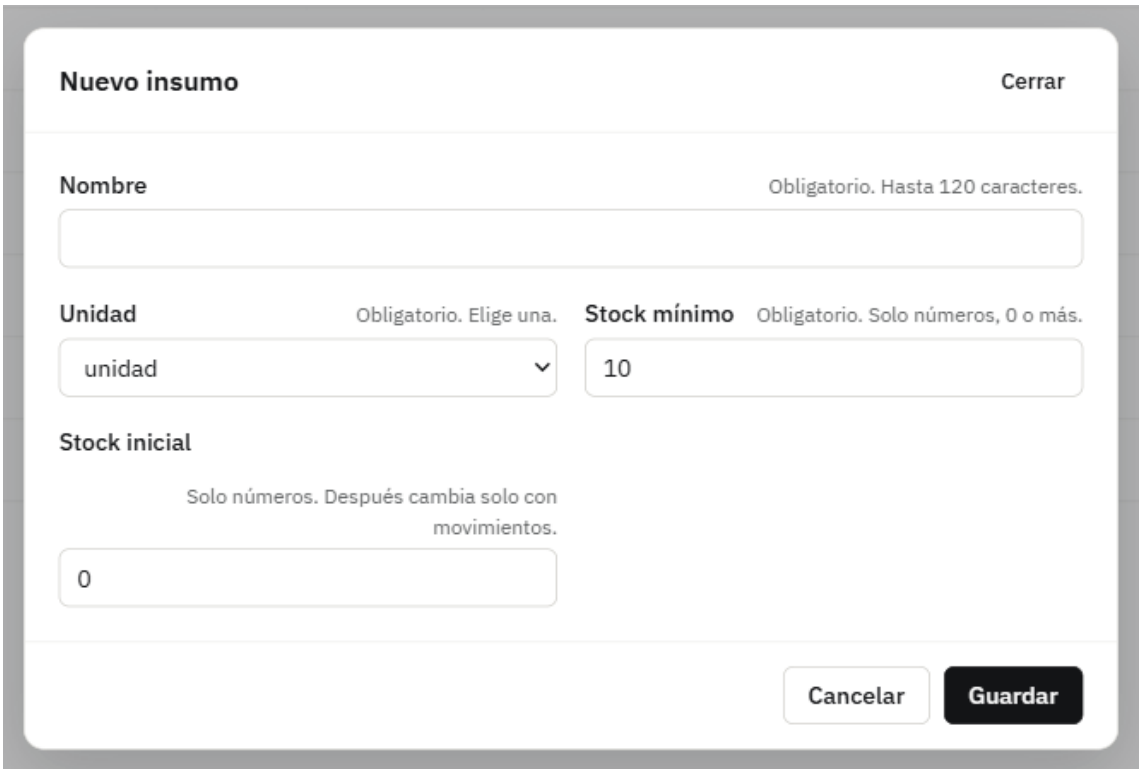


Figura 20. Nuevo insumo: la unidad se elige de una lista y los stocks solo aceptan números

5.3 Personal y Usuarios en una sola pestaña

Antes había tres pestañas que se pisaban entre sí: «Registrar personal», «Personal» y «Usuarios y roles». Ahora hay una sola, «Personal y usuarios», que muestra todas las cuentas del sistema; al pulsar «Agregar persona» se abre el formulario de personal, que crea de una vez la ficha y la

cuenta con la que entra. El cargo sale del rol que se le da, así que se quitaron los campos Cargo y Turno, y también la pestaña de Facturación, que no se usaba.

5.4 Filtros del tablero de habitaciones

El tablero del panel mostraba solo 50 de las 70 habitaciones, porque la API devuelve como máximo 50 por página y el tablero pedía una sola. Ahora pide todas las páginas y muestra las 70. Además se agregaron dos filtros: uno por piso y otro por estado, con el número de habitaciones de cada grupo entre paréntesis.

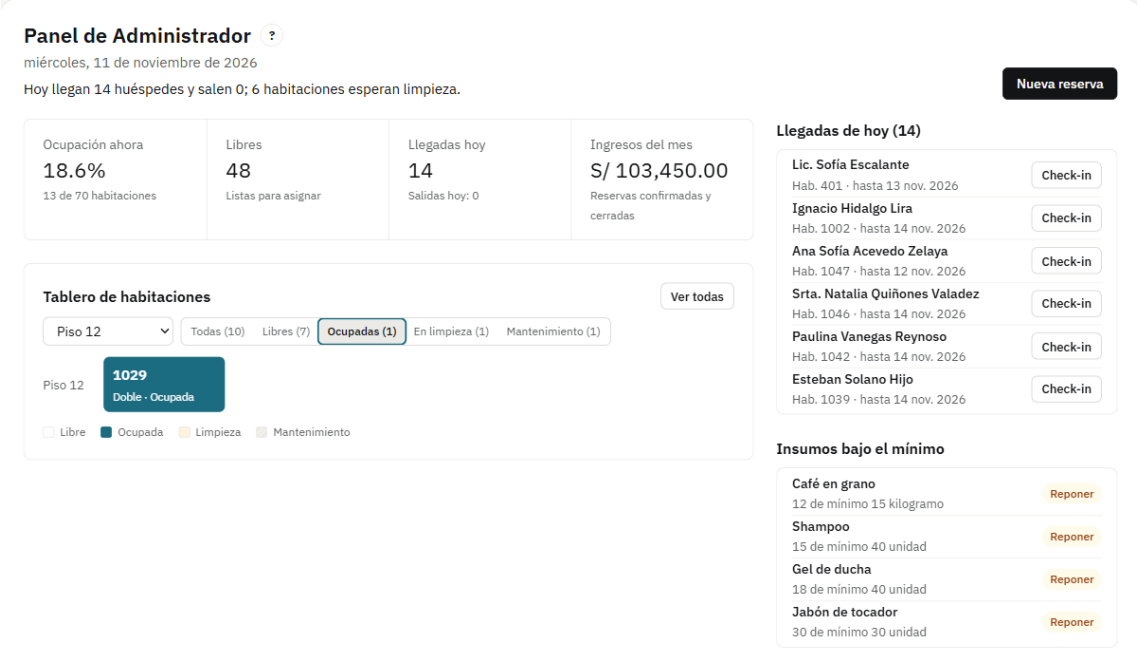


Figura 21. Tablero filtrado por piso 12 y por habitaciones libres

5.5 Despliegue en Render

El proyecto está publicado en Render con tres servicios: una base PostgreSQL, la API en un contenedor Docker y la web como sitio estático. Las migraciones corren al arrancar y los datos de demostración se siembran con una variable, porque el plan gratuito no da consola. La vista del programador muestra ahora el servicio, la rama y el commit desplegados; esos valores se leen desde la configuración y no del entorno, porque en producción la configuración va en caché y de otro modo llegarían vacíos.

El despliegue dejó dos lecciones que quedaron anotadas: la web necesita el dominio público de la API, no el nombre interno que Render usa entre servicios, y el build de la web necesita Node 22, porque la versión por defecto es anterior a la que pide Vite.

6. Pruebas de carga

6.1 Cómo se midió y por qué cambió el método

Las pruebas anteriores apuntaban a la API por localhost:8000. Con cientos de clientes a la vez, el 94 % de las peticiones fallaba con «connection refused» y la prueba no terminaba de medir nada. Se revisó primero Nginx: la cola de conexiones estaba en su valor por defecto de 511, así que se subió a 4096. No cambió nada, seguían 3661 conexiones rechazadas.

La causa era otra. En Windows, Docker Desktop no expone el puerto directamente: pone un proxy intermedio que traduce localhost:8000 hacia el contenedor, y ese proxy aguanta unos cientos de conexiones simultáneas. La prueba estaba midiendo ese puente, no la aplicación. Para comprobarlo se corrió la misma prueba de 500 clientes dentro de la red de Docker, hablando con Nginx directamente.

Tabla 18. La misma prueba de 500 clientes, por dos caminos distintos

Por dónde entra k6	Conexiones rechazadas
localhost:8000 (a través de Windows)	3.661
http://nginx (dentro de la red de Docker)	0

Desde entonces las pruebas se corren con tests/k6/dentro.sh, que ejecuta k6 dentro de la red de Docker. Antes de cada corrida, preparar.sh enciende los contenedores, sube los límites de peticiones (si no, los 429 contarían como errores), siembra los clientes de prueba y calienta la API; al terminar, restaurar.sh la devuelve a su configuración normal.

```
bash tests/k6/preparar.sh 100 --limpiar
bash tests/k6/dentro.sh clientes-1000-reservas.js CLIENTES=100 RESERVAS=5000 BUSCAR=una-vez
bash tests/k6/restaurar.sh
```

Hubo que vaciar el calendario antes de medir. Las pruebas anteriores habían dejado 25.187 reservas en la ventana de un año que usa el escenario, es decir el 98,6 % de ocupación: con el hotel lleno las reservas nuevas chocaban entre sí y el resultado dejaba de medir el sistema. Con la ventana libre, las 70 habitaciones alcanzan de sobra, porque 5000 reservas de una noche son el 20 % de las 25.550 noches-habitación disponibles. No hizo falta crear habitaciones nuevas.

Tabla 19. Entorno de las pruebas

Elemento	Configuración
Equipo	Laptop del equipo, Windows 11
API	Nginx + PHP-FPM en Docker, 20 procesos PHP, OPcache activo
Base de datos	MySQL 8 en Docker
Generador de carga	k6 dentro de la red de Docker (grafana/k6), apuntando a http://nginx
Corte de Nginx	`fastcgi_read_timeout 10s`: pasados 10 s, la petición se corta con 504

6.2 Prueba de estrés: 100 clientes reservando sin pausa

Esta es una prueba de estrés, no de carga normal. Cien clientes del portal reservan a la vez y sin descanso: apenas termina una reserva empiezan la siguiente, sin los segundos que una persona real tarda en elegir fechas y habitación. Es una carga deliberadamente mayor que la del uso real, y por eso lo que se pone a prueba no es la velocidad sino si el sistema se rompe, se traba o se equivoca cuando lo exigen por encima de lo normal.

Cada cliente hace el recorrido completo del portal: inicia sesión, consulta los tipos de habitación libres y registra su reserva. La prueba termina cuando se completan todas las reservas pedidas, no a los X segundos, así que mide cuánto aguanta el sistema haciendo un trabajo fijo bajo presión. Se corrió dos veces para ver si la presión se acumula: primero 1000 reservas y, después de borrarlas para partir del mismo calendario, 5000, es decir cinco veces más trabajo seguido.

Tabla 20. Resultado de las dos corridas

Medida	1000 reservas	5000 reservas
Clientes a la vez	100	100
Tiempo total	34,9 s	2 min 20 s
Reservas creadas	1.000 de 1000	5.000 de 5000
Reservas por segundo	28,7	35,6
Peticiones con error	0,00 %	0,00 %
Conexiones rechazadas	0	0
Mediana por reserva	3,19 s	2,76 s
p95 por reserva	4,27 s	3,30 s

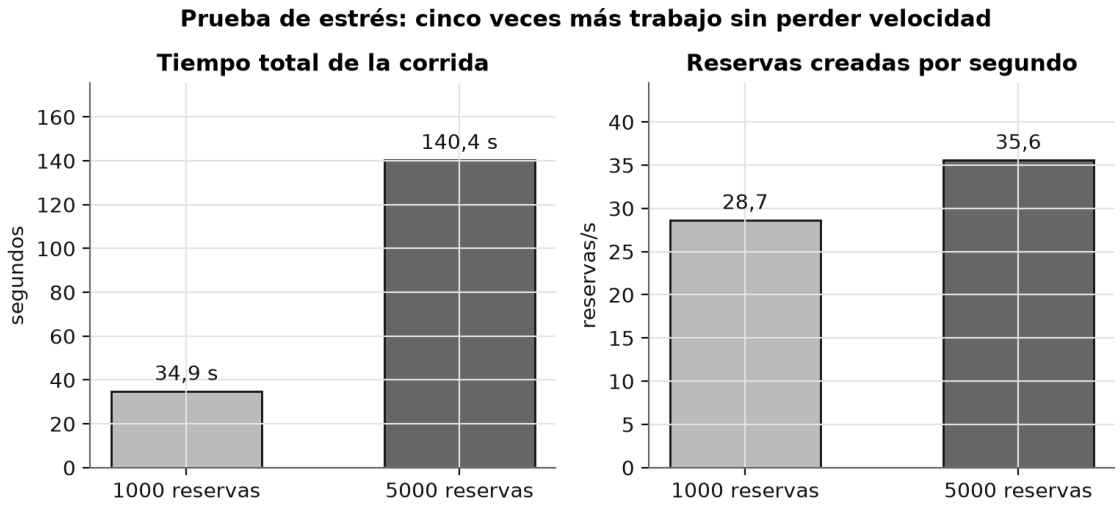


Figura 22. Lo que el sistema sostiene bajo estrés: cinco veces más trabajo seguido, sin perder velocidad

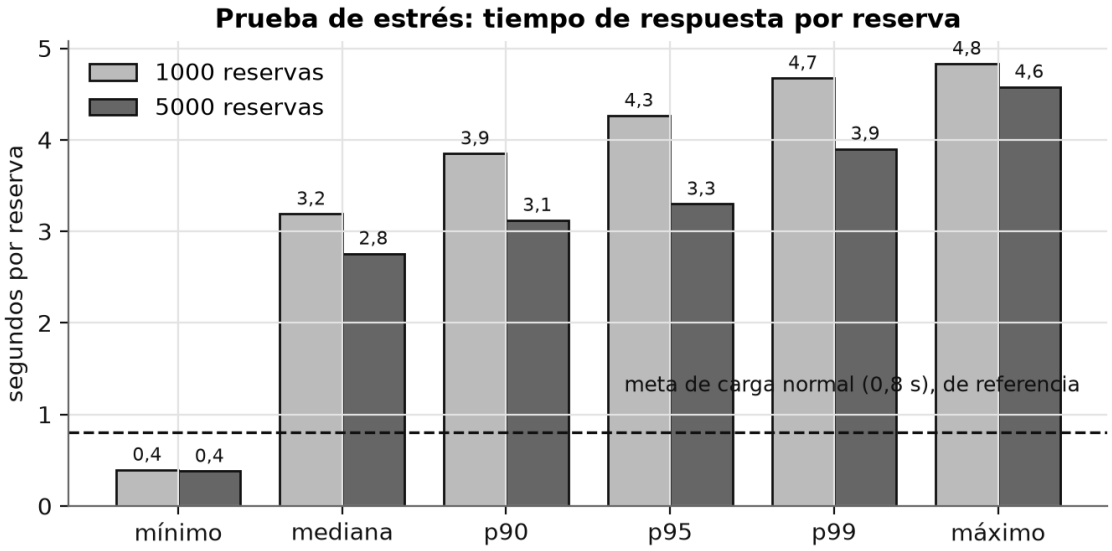


Figura 23. Tiempo de respuesta bajo estrés; la línea de 0,8 s es la meta de carga normal, puesta como referencia

Interpretación. Una prueba de estrés se aprueba o se reprueba por tres cosas: si el sistema aguanta sin caerse, si los datos siguen siendo correctos y si la degradación es ordenada. En las tres el resultado es bueno.

Aguantó. Ninguna de las dos corridas tuvo un solo error ni una conexión rechazada: 0,00 % de peticiones fallidas sobre 1003 y 5003 peticiones. La API no devolvió ni un 500 ni un 503, y no hubo que reiniciar nada. El sistema no se cae cuando lo exigen de más; se limita a tardar más en responder.

Los datos quedaron bien. Es lo más importante de esta prueba. Los 100 clientes compiten por las mismas habitaciones y las mismas fechas, que es justo la condición en la que un sistema mal hecho vende dos veces la misma habitación. Se registraron las 1000 y las 5000 reservas pedidas, sin una sola perdida ni duplicada: el bloqueo por habitación (lockForUpdate) resiste la competencia real, no solo la del papel.

La presión no se acumula. Al quintuplicar el trabajo, el sistema no se fue degradando: al contrario, la corrida larga fue más rápida por reserva (p95 de 3,30 s frente a 4,27 s, y 35,6 reservas por segundo frente a 28,7). Si hubiera una fuga de memoria, una conexión que no se devuelve o una cola que se va llenando, la segunda corrida habría sido peor que la primera y se habría ido frenando. No pasó, así que el sistema se sostiene en el tiempo bajo estrés.

Sobre el tiempo de respuesta: cada reserva tardó entre 3 y 4 segundos, muy por encima de los 0,8 s de la meta. Esa meta es de carga normal y aquí aparece solo como referencia, porque exigirla en una prueba de estrés sería como medir el consumo de un auto a fondo y compararlo con el de ciudad. La cifra que sí sale de esta prueba es otra: bajo estrés el sistema sostiene entre 29 y 36 reservas por segundo. Con la pausa de 6 segundos de un cliente real, el mismo sistema cumple la meta de 0,8 s, como se midió en la entrega anterior.

6.3 Prueba de resistencia: 5000 usuarios entrando en bloques de 100

Si la anterior medía qué aguanta el sistema haciendo un trabajo fijo, esta mide hasta dónde llega antes de romperse. Entran 5000 usuarios de recepción en bloques de 100, uno por segundo, hasta completar el minuto, y cada uno hace lo que haría en su turno: 70 % de consultas, 20 % de inicios de sesión y 10 % de reservas.

Tabla 21. Resultado con 5000 usuarios

Medida	Valor
Usuarios alcanzados	5.000
Duración de la corrida	1 min 52 s (el minuto de subida más la cola de peticiones colgadas)
Peticiones enviadas	36.878
Peticiones con error	98,64 %
Operaciones correctas	410 de 35.337 (1,16 %)
Mediana de respuesta	3,00 s
p95 de respuesta	10,00 s
Conexiones rechazadas	0

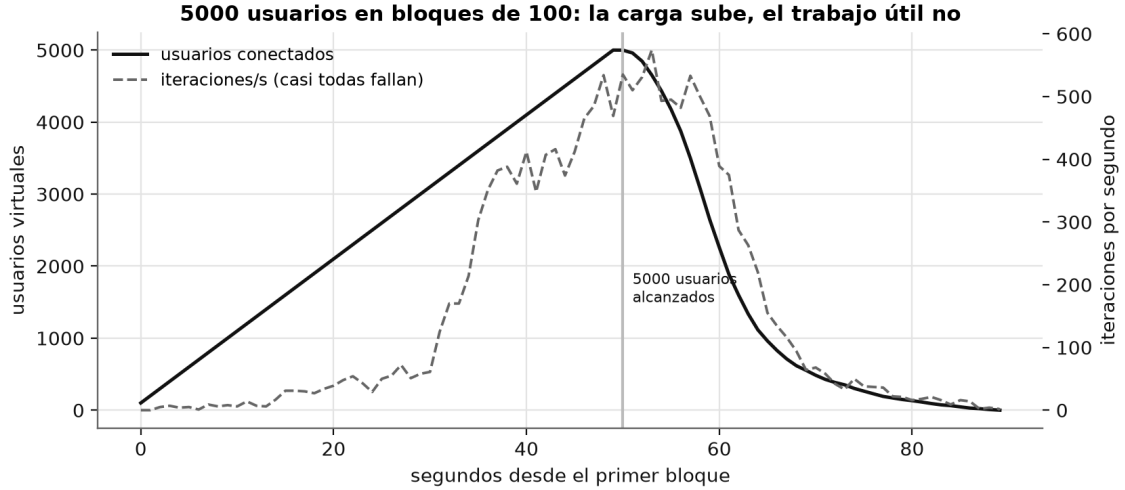


Figura 24. La carga sube de 100 en 100 hasta 5000; las iteraciones por segundo suben con ella, pero casi todas fallan

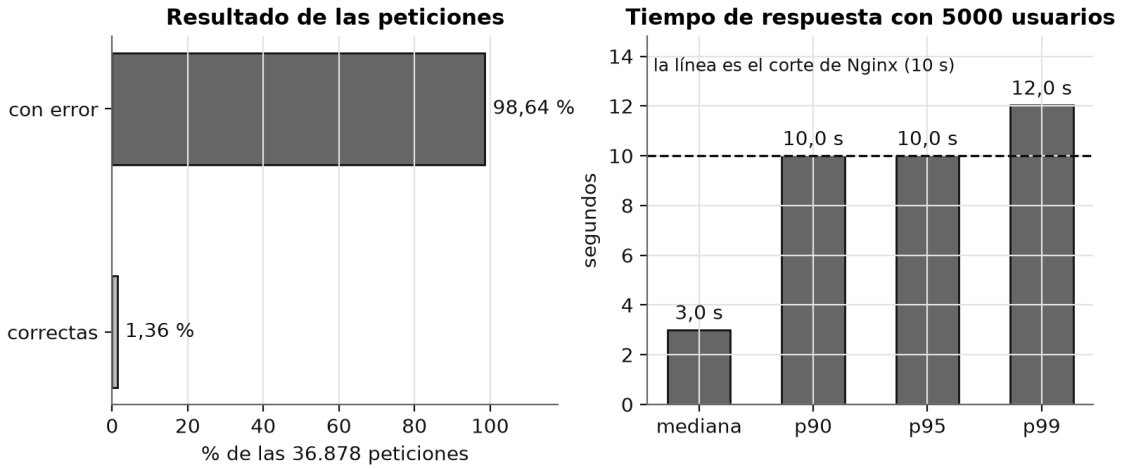


Figura 25. Resultado de las peticiones y tiempo de respuesta con 5000 usuarios encima

Interpretación. El sistema no soporta 5000 usuarios simultáneos en esta laptop, y el resultado dice exactamente por qué. El contenedor tiene 20 procesos de PHP: cada uno atiende una petición a la vez, así que con miles de usuarios encima la cola crece hasta que las peticiones superan los 10 segundos y Nginx las corta con un 504. El p95 clavado en 10 s es justamente ese corte.

Hay un detalle del gráfico que conviene no leer al revés. Las iteraciones por segundo suben hasta cerca de 570 al final de la corrida, pero no son trabajo hecho: son iteraciones que fallan rápido, porque una petición rechazada vuelve antes que una atendida. El dato real es el otro: de 35.337 comprobaciones, solo 410 resultaron correctas. Con 5000 usuarios encima, la API alcanzó a atender unas 410 operaciones.

También es importante que no hubo ninguna conexión rechazada. Eso confirma que el método de medición ya es correcto: lo que falla es la capacidad de la máquina, no el camino hasta ella.

6.4 Dónde está el techo

Juntando esta corrida con las anteriores se puede ubicar el punto de quiebre con bastante precisión. Hasta 600 usuarios simultáneos el sistema responde sin errores; a partir de 700 falla entre el 85 % y el 100 % de las peticiones de cada bloque.

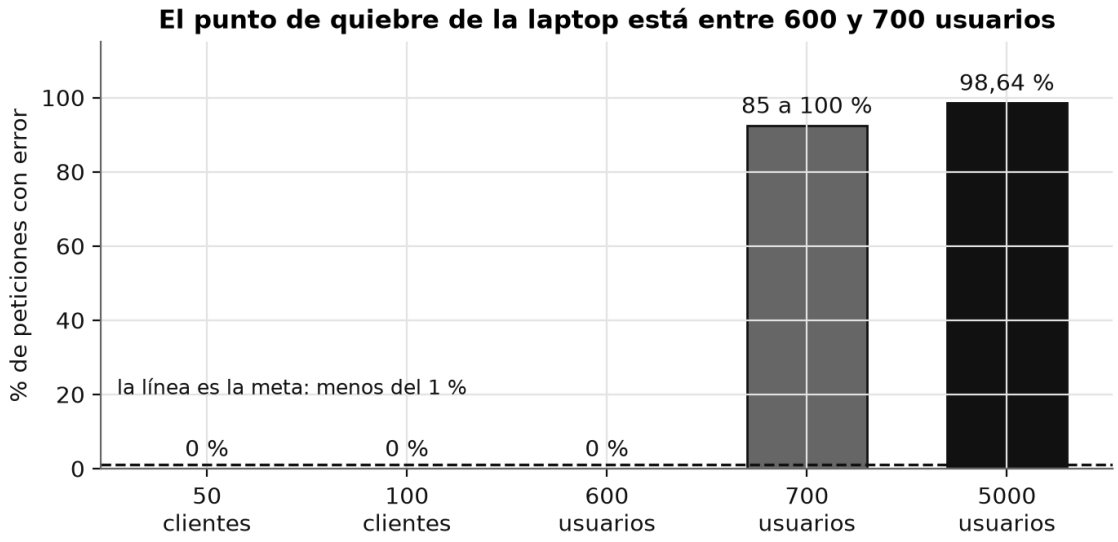


Figura 26. Porcentaje de peticiones con error según cuántos usuarios hay a la vez; con 700 el error varió por bloque entre 85 % y 100 %

Tabla 22. Qué aguanta la laptop

Carga	Resultado
50 a 100 clientes del portal	Sin errores; es el rango que da resultados útiles para medir
Hasta 600 usuarios de recepción	Sin errores
Desde 700 usuarios	Entre 85 % y 100 % de errores por bloque: la cola supera los 10 s y Nginx corta
5000 usuarios	98,64 % de errores

Qué hacer con esto. El límite no está en el código sino en la capacidad de una sola máquina, y por eso la corrección no es programar distinto sino repartir la carga. El plan de infraestructura ya escrito en Terraform lo resuelve con varias instancias EC2 detrás de un balanceador, con Auto Scaling de 1 a 3 instancias, la base en RDS y Redis para caché y colas; en cada instancia habría que subir el número de procesos de PHP según su CPU. Mientras el servidor no exista, estos números describen el techo de una laptop, no el del sistema.

7. Caídas e incidentes del periodo

Cada caída se registra en docs/incidentes.md con su motivo y su solución, aunque sea en local o durante una prueba. Estos son los del 10 de noviembre, el día del despliegue en Render y de las pruebas finales.

Tabla 23. Incidentes registrados el 10/11/2026

Fecha	Entorno	Qué falló	Motivo	Solución	Tiempo
2026-11-10	Local (desarrollo, MySQL)	php artisan migrate (columnas del tutorial) falló con SQLSTATE 2002; docker ps no conectaba con el motor	Docker Desktop estaba apagado otra vez (igual que el 28/10); un primer Start-Process no lo dejó corriendo	Se volvió a abrir Docker Desktop, docker start legrandhotel-mysql-1 y la migración corrió bien. Pest no se afectó (SQLite en memoria)	~8 min
2026-11-10	Local (stack Docker: Nginx + PHP-FPM)	/api/v1/health devolvió 503 SERVICE_UNAVAILABLE dos veces seguidas	Recién encendido Docker, PHP-FPM tardó en responder la primera petición (arranque en frío, sin OPcache caliente) y Nginx cortó por tiempo: «upstream timed out … fastcgi://…:9000» en docker logs legrandhotel-nginx-1	Sin cambios: a los ~2 min respondió 200 con base y caché «ok». Antes de una prueba k6, hacer una petición de calentamiento	~2 min
2026-11-10	Local (prueba k6, 1000 usuarios en bloques de 100)	La API respondió 503/504 al 78% de las peticiones; p95 de 11 s	La laptop llega a ~40–60 pet./s (20 procesos PHP-FPM, login con bcrypt); con más usuarios las peticiones esperan más de 10 s y Nginx corta («upstream timed out»). La base siguió sana	Ninguno en local (es el límite de la laptop). En AWS: más CPU, varias instancias con Auto Scaling, RDS y Redis. Detalle en docs/pruebas/README.md	~4 min (durante la prueba)
2026-11-10	Local (prueba k6)	La primera corrida de 1000 usuarios se detuvo en setup() con 401 al iniciar sesión	Las 50 cuentas de carga se habían creado antes con otra clave y LoadTestUsersSeeder no la actualizaba (insertOrIgnore)	El seeder ahora hace upsert y vuelve a poner la clave de la prueba	~2 min
2026-11-10	Local (prueba k6, 1000 usuarios en 1 minuto)	Desde el bloque de 700 usuarios la API respondió 503/504 (85–100% de errores por bloque); 75% en toda la corrida	Mismo límite de la laptop (~45 pet./s): con 700 usuarios las peticiones esperan más de 10 s y Nginx corta. Hasta 600 usuarios no hubo errores	Ninguno en local; se repetirá en AWS con Auto Scaling. Tabla por bloque en docs/pruebas/README.md	~30 s (durante la prueba)
2026-11-10	Local (stack Docker)	Después de las pruebas k6 la web quedaba en «Demasiadas peticiones» y la API respondía 429 a todo, aun sin carga	api/.env tenía RATE_LIMIT_API=, RATE_LIMIT_LOGIN= y RATE_LIMIT_REGISTER= vacías: (int) env('RATE_LIMIT_API', 60) no usa el valor por defecto con una línea vacía y el límite quedaba en 0 (cabecera X-RateLimit-Limit: 0)	config/hotel.php usa el valor por defecto si la variable está vacía (prueba RateLimitConfigTest); api/.env vuelve a 5 / 3 / 60 como .env.example	~10 min

Fecha	Entorno	Qué falló	Motivo	Solución	Tiempo
2026-11-10	Render (producción)	La web mostraba la pantalla 503 «No se pudo contactar con el servidor» aunque la API respondía 200 y el CORS era correcto	render.yaml armaba VITE_API_URL con fromService … property: host, que devuelve el nombre interno del servicio (legrandhotel-api), no el dominio público. La web pedía https://legrandhotel-api/api/v1/…, que el navegador no resuelve, y fallaba antes de enviar la petición (ERR_BLOCKED_BY_CLIENT): por eso no aparecía nada en la pestaña de red. Se encontró interceptando XMLHttpRequest.open en la página	Se usa el dominio público completo (https://legrandhotel-api.onrender.com/api/v1) en el buildCommand y se quitó API_HOST. Regla: property: host solo sirve entre servicios, nunca para el navegador	~1 h (desde el despliegue hasta detectarlo)
2026-11-10	Local (prueba k6, 500 clientes)	94% de las peticiones fallaban con «connection refused» y solo se creaban 246 reservas de 5000	No era Nginx: subir listen 80 backlog=4096 no cambió nada (seguían 3661 rechazos). En Windows, Docker Desktop publica localhost:8000 con un proxy propio que aguanta unos cientos de conexiones simultáneas; con 500 clientes de golpe las rechaza. Prueba: la misma corrida dentro de la red de Docker (http://nginx) dio 0 conexiones rechazadas	Nuevo tests/k6/dentro.sh, que corre k6 dentro de la red de Docker. Dentro ya no hay rechazos, pero aparece el límite real: 20 procesos PHP-FPM, así que con 500 clientes las peticiones pasan de 10 s y Nginx corta (504). Para medir la API hay que usar dentro.sh y no más de ~50–100 clientes en esta laptop	~1 h

8. Conclusiones y pendientes

El sistema está desplegado y funcionando en internet, con todas las pantallas documentadas y un tutorial que explica cada una desde dentro de la propia aplicación. Las pruebas de carga ya miden lo que deben medir, después de descubrir que el problema no estaba en la aplicación sino en el puente de red de Docker Desktop en Windows.

Sobre el rendimiento, la conclusión es doble. El sistema es correcto bajo presión: con 100 clientes compitiendo por las mismas fechas registró 5000 reservas sin perder ni duplicar ninguna. Y es limitado en capacidad: una sola máquina con 20 procesos de PHP se queda corta a partir de unos 700 usuarios simultáneos. Lo primero es mérito del código; lo segundo se resuelve con infraestructura.

Pendientes

- Crear el entorno en AWS con el Terraform ya escrito y desplegar con el pipeline.
- Repetir en ese servidor las pruebas de 1000 y 5000 usuarios, desde máquinas generadoras y no desde la laptop.
- Configurar los respaldos automáticos en AWS (snapshots de RDS y copia diaria a S3).
- Registrar en Google el dominio definitivo para el ingreso con correo.